

Understanding Diffusion Models: A Complete Guide

What Are Diffusion Models?

In one sentence: Diffusion models generate images by learning to reverse a noise-adding process, starting from **pure noise** and gradually removing it to create realistic images.

Think of it like this: imagine you have a **photo** that gets damaged a little bit each day for 1000 days until it's completely destroyed. A diffusion model learns to perfectly reverse this damage, step by step, until it can take any destroyed photo (or pure noise) and restore it into a beautiful, coherent image.

The Core Insight: Why Add Noise to Remove It?

This might seem backwards at first - why would we deliberately **destroy images** just to learn how to fix them? The answer lies in a powerful mathematical insight:

Learning to remove noise is much easier than learning to create images from scratch.

-  **Hard problem:** "Generate a realistic image of anything"
-  **Easier problem:** "Remove this specific noise from this image"

By breaking down image generation into thousands of tiny denoising steps, we transform an impossible problem into a manageable one.

The Magic of the Process

Forward Process (Training Data Creation)

Beautiful Image → Add noise → Slightly noisy → More noisy → ... → Pure noise

x_0	x_1	x_2	x_3	x_{1000}
-------	-------	-------	-------	------------

Reverse Process (Image Generation)

Pure noise → Denoise → Less noisy → Cleaner → ... → Beautiful Image

x_{1000}	x_{999}	x_{998}	x_{997}	x_0
------------	-----------	-----------	-----------	-------

The model learns: *"If I see this level of noise at step 547, what should the image look like at step 546?"*

Why This Approach Works So Well

1. Mathematical Foundation

- Built on well-understood Gaussian processes
- Each step is a simple, learnable transformation
- Provably converges to the data distribution

2. Training Stability

- No adversarial training (unlike GANs)
- Each step has a clear, well-defined target
- Gradients flow smoothly through the process

3. Quality Control

- Generate slowly but surely - each step improves the image
- Can stop early for faster (but lower quality) results
- Can run longer for higher quality

Diffusion Based Models

- Popular services include
 - **DALL.E2**
 - **Google ImageGen**
 - **StableDiffusion**
- Origins are from a paper published in **2020**
 - Denoising - Diffusion Probabilistic Model (DDPM).

We're are going to cover some elements of **stable diffusion**, the main components are

- Variational Autoencoder (VAE) for transforming images

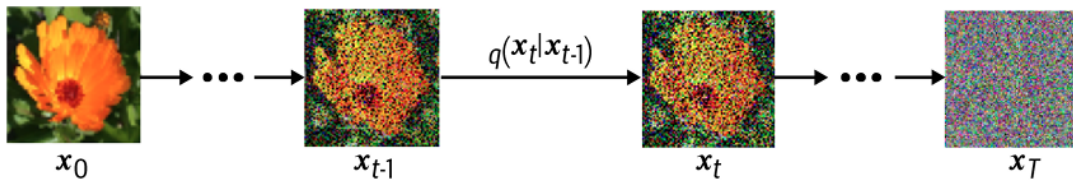
- **CLIP encoder** for transforming text → VIT VisionTransformers
- **U-NET** for estimating the noise
- **Scheduler** for determining how to remove noise

Introduction to Denoising Diffusion Models (DDMs)

Denoising Diffusion Models (DDMs) are a class of generative models that create images by progressively refining random noise into coherent and realistic outputs. The process involves two main stages:

Core Concept of DDMs

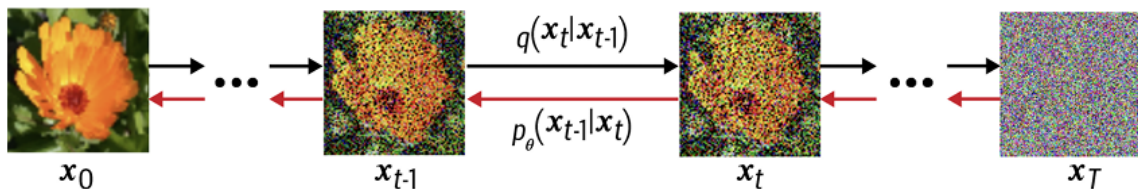
1. Forward Process (Diffusion):



- A **scheduler** (e.g., linear, cosine, or offset cosine schedules) defines how noise is added progressively to an image, step by step, until it becomes pure noise.

2. Reverse Process (Denoising):

- A **U-Net** learns to reverse this process by estimating the noise added at each step and removing it iteratively to recover the original image.



How the U-Net and Scheduler Work Together

1. Scheduler:

- Provides the noise schedule, which determines:

- How much noise to add during training (forward diffusion).
- How much noise to remove during generation (reverse diffusion).
- It calculates the noise and signal rates (`noise_rates` , `signal_rates`) for each diffusion step.

2. U-Net:

- Takes the noisy image and noise variance (via embeddings) as input.
- Predicts the noise component in the image at each step.
- The predicted noise is used to denoise the image iteratively, guided by the scheduler.

Why the Combination Defines a DDM

- The **scheduler** ensures that the diffusion process follows a mathematically defined trajectory of adding and removing noise.
- The **U-Net** serves as the predictive engine, learning to reverse the diffusion process step-by-step.

Together, they form the essence of a DDM:

- The scheduler models the forward and reverse noise processes.
- The U-Net learns how to denoise effectively within this framework.

1. Diffusion Schedules and Why Define Multiple Ones?

Diffusion schedules are critical components in denoising diffusion models (DDMs). They define how noise is **added** to an image over a series of steps during the **forward diffusion process** and how the **model removes it during the reverse diffusion process**. The schedule determines the rate at which signal (useful information) is lost and noise is added over time.

By defining multiple schedules, we can explore and optimize how the noise and signal rates evolve over time, which can impact:

- **Training Stability:** Different schedules can make the learning process **more stable or challenging**.
- **Generation Quality:** The quality of **generated images** often depends on how well the schedule balances noise and signal.

- **Model Interpretability: Different schedules** may reveal specific patterns or behaviors in the denoising process.
-

Understanding the Code

1. `linear_diffusion_schedule` :

- Adds noise linearly over time, starting with a small rate (`min_rate`) and increasing it to a maximum rate (`max_rate`).
- **Key Components:**
 - `betas` : The rate at which noise is added at each step.
 - `alphas` : The remaining signal after noise is added (`1 - betas`).
 - `alpha_bars` : Cumulative product of `alphas` , representing the overall retained signal across all steps.
 - `signal_rates` and `noise_rates` : Define the tradeoff between retained signal and added noise.
- **Use Case:** Simple, linear schedules are easy to understand and implement, serving as a baseline for experiments.

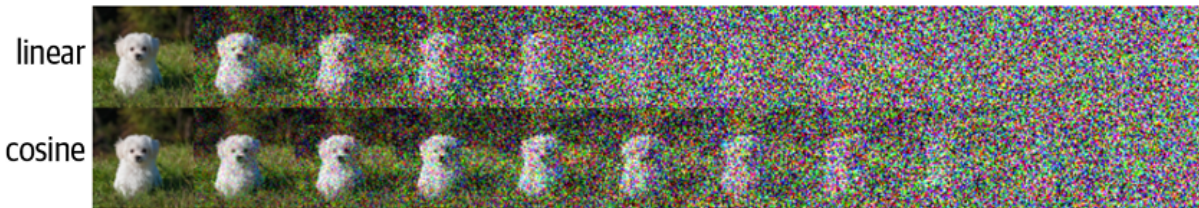
2. `cosine_diffusion_schedule` :

- Uses a cosine function to gradually reduce the signal and increase noise over time.
- **Key Components:**
 - `signal_rates` : Follow a cosine curve to retain more signal in the earlier steps.
 - `noise_rates` : Follow a sine curve, increasing noise gradually.
- **Use Case:** Cosine schedules are smoother than linear schedules, which can improve training stability and generation quality.

3. `offset_cosine_diffusion_schedule` :

- Modifies the cosine schedule with adjustable start (`max_signal_rate`) and end (`min_signal_rate`) points.
- **Key Components:**
 - `diffusion_angles` : Controls the transition from signal to noise, derived from the adjusted cosine curve.

- **Use Case:** Adds flexibility to the cosine schedule, allowing control over how much signal is retained or noise is added at the start and end.



Why Define Multiple Schedules?

1. Experimentation:

- Each schedule behaves differently. Comparing multiple schedules allows us to evaluate their impact on training efficiency, model convergence, and image quality.

2. Tailored Applications:

- Some schedules may perform better on certain datasets or tasks. For example:
 - A **linear schedule** is straightforward and may work well for simple datasets.
 - A **cosine schedule** could provide smoother transitions, useful for complex datasets.
 - An **offset cosine schedule** allows fine-tuning of noise addition, making it suitable for datasets with specific characteristics.

3. Insights into Model Behavior:

- Visualizing signal and noise rates for each schedule (as in the plotted graph) helps understand how the model processes information over time.

Visualization

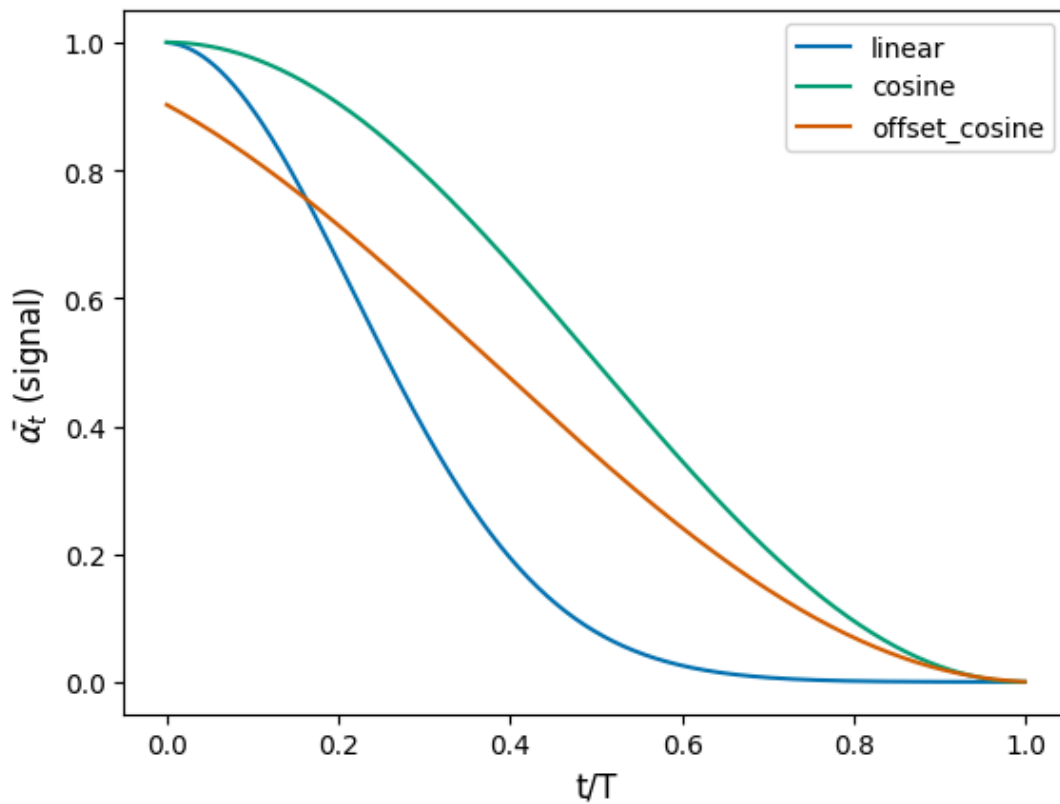
The graph shows how the signal (α_t , or `signal_rates`) evolves over time (t/T) for each schedule:

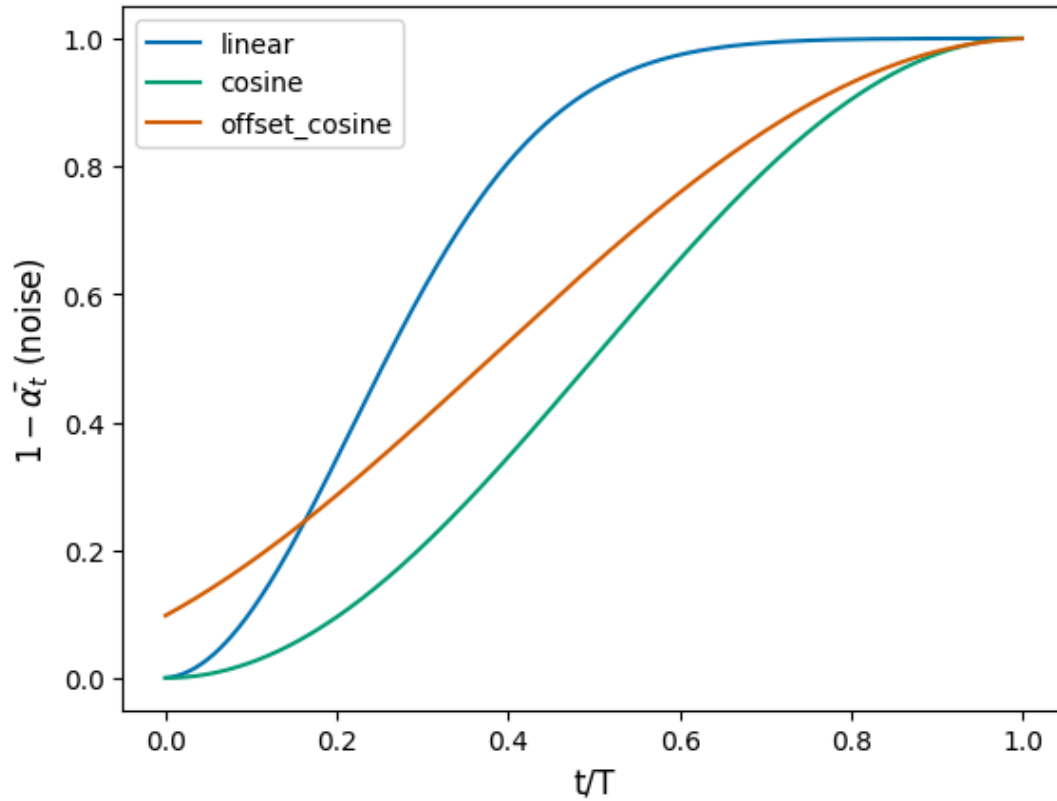
- **Linear:** Signal decreases linearly, reaching 0 at the end.

- **Cosine:** Signal decreases smoothly, following a cosine curve.
- **Offset Cosine:** Signal decreases smoothly but starts and ends at customized rates.

Interpretation:

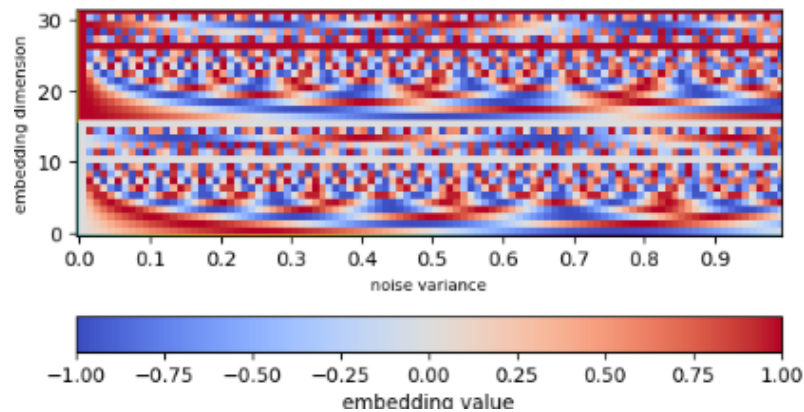
- The smoother curves (cosine and offset cosine) might retain more useful information in the earlier steps, allowing the model to learn more effectively.
- The linear schedule, while simpler, might lose too much information early on, depending on the task.





2. Sinusoidal Embedding Visualization

The purpose of this lab section is to understand **sinusoidal embeddings** and visualize how they encode values across different noise levels. Here's a breakdown of the code and the underlying concepts:



What is a Sinusoidal Embedding?

Sinusoidal embeddings are a way to encode continuous values (like time steps or noise levels) into a **higher-dimensional space** using sine and cosine functions. These embeddings:

1. Help capture periodic patterns and relationships.
2. Are commonly used in models like Transformers and Diffusion Models to encode continuous values effectively.

In the context of **denoising diffusion models (DDMs)**:

- The sinusoidal embedding maps the noise variance (or time step) into a high-dimensional representation, allowing the model to understand "**when**" in the diffusion process it is operating.

```
def sinusoidal_embedding(x):
    frequencies = tf.exp(
        tf.linspace(
            tf.math.log(1.0),
            tf.math.log(1000.0),
            NOISE_EMBEDDING_SIZE // 2,
        )
    )
    angular_speeds = 2.0 * math.pi * frequencies
    embeddings = tf.concat(
        [tf.sin(angular_speeds * x), tf.cos(angular_speeds * x)], axis=3
    )
    return embeddings
```

- **Inputs:**

- `x`: Noise variance or time step, normalized between 0 and 1.

- **Process:**

1. **Frequencies:** Generate exponentially spaced frequencies between `1.0` and `1000.0`. These define how fast the sine/cosine functions oscillate.
2. **Angular Speeds:** Compute angular frequencies (`2π * frequency`).
3. **Embedding:** Apply sine and cosine transformations to the input, then concatenate the results along the last dimension to form the embedding.

- **Output:**
 - A sinusoidal embedding of shape `[batch_size, height, width, NOISE_EMBEDDING_SIZE]` .

Why Sinusoidal Embeddings?

1. Capturing Relationships:

- The periodic nature of sine and cosine functions ensures that embeddings can represent relationships across noise levels.

2. Smooth Encoding:

- Embeddings provide a smooth transition as the noise variance changes, making them ideal for continuous processes like diffusion.

3. Dimensional Representation:

- By encoding noise variance into a higher-dimensional space, the model gets rich features that help it better understand the diffusion process.

3. The U-Net Architecture

U-Net is a convolutional neural network architecture originally designed for biomedical image segmentation. In the context of **denoising diffusion models (DDMs)**, U-Net is used to predict **clean** images from **noisy inputs** by leveraging its ability to combine **low-level** and **high-level features** through its encoder-decoder structure.

Key Components of the U-Net

1. Residual Block (`ResidualBlock`)

- A building block of the U-Net that incorporates skip connections.
- **Purpose:**
 - Helps stabilize training and improve gradient flow by directly passing the input (`residual`) alongside processed features.
- **Steps:**
 1. **Batch Normalization:** Normalizes activations to improve convergence.
 2. **Conv2D Layers:** Two 3×3 convolution layers to extract spatial features.

3. **Skip Connection:** Adds the input (`residual`) back to the output of the convolutions.

2. Down Block (`DownBlock`)

- A down-sampling layer used in the encoder of the U-Net.
- **Purpose:**
 - Reduces spatial dimensions (height and width) while increasing feature richness.
 - Captures hierarchical features of the image at different scales.
- **Steps:**
 1. Repeatedly applies `ResidualBlock` to refine features.
 2. Saves intermediate features (`skips`) for use in the decoder.
 3. Down-samples the spatial dimensions using `AveragePooling2D` .

3. Up Block (`UpBlock`)

- An up-sampling layer used in the decoder of the U-Net.
- **Purpose:**
 - Gradually reconstructs the image by combining features from the encoder (`skips`) and up-sampled features.
 - Restores spatial resolution while refining details.
- **Steps:**
 1. Increases spatial dimensions using `UpSampling2D` .
 2. Merges encoder features (`skips`) with the current layer using `Concatenate` .
 3. Refines the combined features with `ResidualBlock` .

Noise Embedding

```
noise_embedding = layers.Lambda(sinusoidal_embedding)(noise_variances)
noise_embedding = layers.UpSampling2D(size=IMAGE_SIZE, interpolation="nearest")(noise_embedding)
```

```
x = layers.Concatenate()([x, noise_embedding])
```

- **Purpose:**

- Encodes noise variance into a high-dimensional representation using the sinusoidal embedding.
- Provides the model with information about the noise level, enabling it to learn how to denoise progressively.

- **Process:**

1. `sinusoidal_embedding` : Maps noise variance to an embedding.
2. `UpSampling2D` : Matches the embedding's spatial resolution to the input image.
3. `Concatenate` : Combines the noise embedding with the image features.

Building the U-Net

Encoder (Down Blocks):

```
x = DownBlock(32, block_depth=2)([x, skips])  
x = DownBlock(64, block_depth=2)([x, skips])  
x = DownBlock(96, block_depth=2)([x, skips])
```

- Reduces the spatial dimensions progressively.
- Captures hierarchical features at multiple resolutions.

Bottleneck (Residual Blocks):

```
x = ResidualBlock(128)(x)  
x = ResidualBlock(128)(x)
```

- Processes the lowest resolution representation of the image.
- Extracts the most abstract features.

Decoder (Up Blocks):

```
x = UpBlock(96, block_depth=2)([x, skips])
x = UpBlock(64, block_depth=2)([x, skips])
x = UpBlock(32, block_depth=2)([x, skips])
```

- Gradually restores spatial resolution while combining high-level (low-resolution) and low-level (high-resolution) features using `skips`.

Final Layer:

```
x = layers.Conv2D(3, kernel_size=1, kernel_initializer="zeros")(x)
```

- Outputs the reconstructed image with 3 channels (RGB).
- `kernel_initializer="zeros"` ensures the initial output is near zero, stabilizing training.

U-Net Summary

- **Input:**
 - `noisy_images`: Noisy image tensor of shape `(IMAGE_SIZE, IMAGE_SIZE, 3)`.
 - `noise_variances`: Scalar noise level for the image, expanded into a high-dimensional embedding.
- **Output:**
 - Denoised image tensor of shape `(IMAGE_SIZE, IMAGE_SIZE, 3)`.
- **Architecture:**
 - Combines features at different resolutions using skip connections.
 - Leverages residual blocks to stabilize training and enhance feature extraction.
- **Why U-Net for DDMs?**
 - Captures global and local image features effectively.
 - Allows the model to denoise progressively by understanding the hierarchical structure of the noise.

4. The Diffusion Model

The `DiffusionModel` class is the heart of the denoising diffusion framework. It encapsulates the core functionality for training, testing, and generating images using a denoising diffusion model (DDM). Below is a detailed explanation of the class and its methods:

Key Components of the Diffusion Model

1. Initialization (`__init__`)

```
def __init__(self):
    super().__init__()
    self.normalizer = layers.Normalization()
    self.network = unet
    self.ema_network = models.clone_model(self.network)
    self.diffusion_schedule = offset_cosine_diffusion_schedule
```

- `normalizer` : Normalizes input images to stabilize training.
- `network` : The main U-Net architecture that predicts noise.
- `ema_network` : An exponential moving average (EMA) version of the U-Net, used for more stable predictions during evaluation.
- `diffusion_schedule` : Specifies the noise-signal schedule (e.g., offset cosine).

2. Training Metrics

```
def compile(self, **kwargs):
    super().compile(**kwargs)
    self.noise_loss_tracker = metrics.Mean(name="n_loss")

@property
def metrics(self):
```

```
return [self.noise_loss_tracker]
```

- Tracks the loss (`noise_loss`) during training.
- Helps monitor the performance of the model.

3. Denormalization

```
def denormalize(self, images):  
    images = self.normalizer.mean + images * self.normalizer.variance**0.5  
    return tf.clip_by_value(images, 0.0, 1.0)
```

- Converts normalized images back to their original scale.
- Clips pixel values to ensure they are within the valid range `[0, 1]`.

4. Denoising

```
def denoise(self, noisy_images, noise_rates, signal_rates, training):  
    if training:  
        network = self.network  
    else:  
        network = self.ema_network  
    pred_noises = network([noisy_images, noise_rates**2], training=training)  
    pred_images = (noisy_images - noise_rates * pred_noises) / signal_rates  
    return pred_noises, pred_images
```

- **Purpose:** Predicts the clean image and noise components from noisy images.
- **Steps:**
 - **Prediction:** Uses either the training network or the EMA network to predict noise.
 - **Reconstruction:** Computes the predicted clean image using the noise prediction.

5. Reverse Diffusion

```
def reverse_diffusion(self, initial_noise, diffusion_steps):
    num_images = initial_noise.shape[0]
    step_size = 1.0 / diffusion_steps
    current_images = initial_noise
    for step in range(diffusion_steps):
        diffusion_times = tf.ones((num_images, 1, 1, 1)) - step * step_size
        noise_rates, signal_rates = self.diffusion_schedule(diffusion_times)
        pred_noises, pred_images = self.denoise(
            current_images, noise_rates, signal_rates, training=False
        )
        next_diffusion_times = diffusion_times - step_size
        next_noise_rates, next_signal_rates = self.diffusion_schedule(
            next_diffusion_times
        )
        current_images = (
            next_signal_rates * pred_images + next_noise_rates * pred_noises
        )
    return pred_images
```

- **Purpose:** Gradually denoises starting from pure noise to generate clean images.
- **Steps:**
 - Predicts the noise and clean image at each step.
 - Combines these predictions with the next step's noise and signal rates to move toward a cleaner image.

6. Image Generation

```
def generate(self, num_images, diffusion_steps, initial_noise=None):
    if initial_noise is None:
        initial_noise = tf.random.normal(
            shape=(num_images, IMAGE_SIZE, IMAGE_SIZE, 3)
        )
    generated_images = self.reverse_diffusion(initial_noise, diffusion_steps)
```



```
generated_images = self.denormalize(generated_images)
return generated_images
```

- **Purpose:** Generates images by running the reverse diffusion process.
- **Input:**
 - `num_images` : Number of images to generate.
 - `diffusion_steps` : Number of steps in the reverse diffusion process.
 - `initial_noise` : Starting point for the reverse diffusion. If not provided, random noise is used.

7. Training Step

```
def train_step(self, images):
    images = self.normalizer(images, training=True)
    noises = tf.random.normal(shape=(BATCH_SIZE, IMAGE_SIZE, IMAGE_SIZE, 3))

    diffusion_times = tf.random.uniform(
        shape=(BATCH_SIZE, 1, 1, 1), minval=0.0, maxval=1.0
    )
    noise_rates, signal_rates = self.diffusion_schedule(diffusion_times)

    noisy_images = signal_rates * images + noise_rates * noises

    with tf.GradientTape() as tape:
        pred_noises, pred_images = self.denoise(
            noisy_images, noise_rates, signal_rates, training=True
        )
        noise_loss = self.loss(noises, pred_noises)

    gradients = tape.gradient(noise_loss, self.network.trainable_weights)
    self.optimizer.apply_gradients(
        zip(gradients, self.network.trainable_weights)
    )
```

```

self.noise_loss_tracker.update_state(noise_loss)

for weight, ema_weight in zip(
    self.network.weights, self.ema_network.weights
):
    ema_weight.assign(EMA * ema_weight + (1 - EMA) * weight)

return {m.name: m.result() for m in self.metrics}

```

- **Purpose:** Implements a single training step.
- **Steps:**
 1. Add noise to the input images.
 2. Train the network to predict the noise.
 3. Compute gradients and update weights.
 4. Update the EMA network for stable evaluation.

8. Test Step

```

def test_step(self, images):
    images = self.normalizer(images, training=False)
    noises = tf.random.normal(shape=(BATCH_SIZE, IMAGE_SIZE, IMAGE_SIZE, 3))
    diffusion_times = tf.random.uniform(
        shape=(BATCH_SIZE, 1, 1, 1), minval=0.0, maxval=1.0
    )
    noise_rates, signal_rates = self.diffusion_schedule(diffusion_times)
    noisy_images = signal_rates * images + noise_rates * noises
    pred_noises, pred_images = self.denoise(
        noisy_images, noise_rates, signal_rates, training=False
    )
    noise_loss = self.loss(noises, pred_noises)
    self.noise_loss_tracker.update_state(noise_loss)

```

```
return {m.name: m.result() for m in self.metrics}
```

- **Purpose:** Implements a single evaluation step.
- **Steps:**
 - Similar to the training step but without gradient updates.

Summary of Functionality

1. Training:

- Adds noise to input images and trains the network to predict the noise, gradually improving the denoising process.

2. Generation:

- Uses reverse diffusion to generate clean images starting from random noise.

3. Evaluation:

- Tracks performance using the test step and noise loss metrics.

4. EMA Network:

- Provides a more stable version of the model for evaluation and generation.

This class ties together all aspects of the denoising diffusion process into a cohesive framework for training and image generation.

QA Section

What kind of images do I need to train this model?

You can train this model on **any type of images** as long as you have a sufficient **quantity** and **quality** of data. The model learns the statistical structure of the training images and generates similar images during inference. Here are some considerations:

- **Diversity:** The dataset should represent the kinds of images you want the model to generate.
 - **Resolution:** Images need to be resized to the `IMAGE_SIZE` parameter (e.g., 64×64 in this case).
 - **Quantity:** The more images you have, the better the model will generalize. For smaller datasets, you can augment images (e.g., rotations, flips) to increase dataset size.
 - **Loss Function:** The loss is based on how well the model predicts noise, so even noisy or low-quality images can be useful if the goal is general image synthesis.
-

Do I still need training and test sets?

Yes, it is essential to split your dataset into **training** and **test sets** to evaluate the model's performance and avoid overfitting. Here's why:

- **Training Set:** Used to train the model, allowing it to learn the diffusion and denoising processes.
- **Test Set:** Used to evaluate the model's ability to generalize to unseen data. It ensures that the model isn't just memorizing the training data.

However, diffusion models are **unsupervised**, so you won't need labels for the images. The train-test split is mainly for validating the generation quality.

Where do I put the prompt?

This model does not support text-based prompting because it is a **basic diffusion model** that generates images without guidance. Prompting requires additional components like:

- **CLIP (Contrastive Language–Image Pretraining):** To encode text prompts and guide image generation.
- **VIT (Vision Transformer):** For fine-grained understanding and alignment of text with image features.

In this case, the model can only generate images based on the statistical patterns in the training data without any guidance or text-based control. If prompting is required, you would need to integrate a pre-trained CLIP model or similar architecture.

Can I guide the generation process?

While this model cannot take textual prompts, you can guide generation to some extent by:

- **Changing the noise schedule:** Modify the `diffusion_schedule` to control the rate of noise addition and removal.
 - **Latent Control:** Alter the starting noise (`initial_noise`) to influence the style or structure of generated images.
-

What exactly are we doing with noise? When do we add noise? When do we remove it?

What are we doing with noise?

Noise plays a central role in both training and generating images in diffusion models. Essentially, noise is used to:

1. **Corrupt the images** during the forward diffusion process. This helps the model learn how to reverse the corruption step by step.
 2. **Guide generation** during the reverse diffusion process, starting from pure noise and iteratively removing it to create a coherent image.
-

When do we add noise?

- **During Training:**
 - Noise is added to the original images incrementally over a series of steps.
 - At each step t , the image becomes progressively more corrupted with noise until it resembles pure Gaussian noise.
 - This is done according to a predefined **noise schedule** (e.g., linear or cosine schedules) that determines how much noise to add at each step.
 - The purpose is to train the model to predict and reverse the noise at any given step.
-

When do we remove noise?

- **During Generation:**

- Starting with pure noise, the reverse diffusion process removes noise step by step.
 - At each step t , the model (U-Net) predicts the noise in the current noisy image. This prediction is subtracted from the noisy image to reconstruct a less noisy version of the image.
 - The process continues iteratively, guided by the same noise schedule, until a clear image is generated.
-